

LC 2556: Disconnect Path in a Binary Matrix

Authored by Arpit Raj, LNMIIT Jaipur | July 28, 2026

Problem Statement & Rules

You can't flip the cells $(0,0)$ and $(m-1, n-1)$.

Tell if a single flip can disconnect the path from $(0,0)$ to $(m-1, n-1)$.

Brute Force Approach

$O((nm)^2)$ — try to change every cell and do BFS every time.

```

class Solution {
public:

    vector<pair<int,int>> dir={{-1,0},{0,-1},{0,1},{1,0}};
    int n;
    int m;

    bool canreach(vector<vector<int>>& temp){
        queue<pair<int,int>> q;
        q.push({0,0});

        while(!q.empty()){
            int r=q.front().first;
            int c=q.front().second;

            q.pop();
            if(r==n-1 && c==m-1) return true;

            for(auto &it:dir){
                int nr=r+it.first;
                int nc=c+it.second;

                if(nr>=0 && nc>=0 && nr<n && nc<m){
                    if(temp[nr][nc]==1){
                        q.push({nr,nc});
                        temp[nr][nc]=0;
                    }
                }
            }
        }

        return false;
    }

    bool isPossibleToCutPath(vector<vector<int>>& grid) {

        n=grid.size();
        m=grid[0].size();
        vector<vector<int>> temp=grid;

        if(!canreach(temp)) return true; //flip ki zarurat hi nahi padi aise hi di
        bool flag=false;
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){

                if((i==0 && j==0) || (i==n-1 && j==m-1)) continue;

                temp=grid;
                flag=false; //we will transform every cell
                            //lets assume disconnect nahi kr skte

                if(temp[i][j]==1){
                    temp[i][j]=0;
                }

                else{
                    continue; //cant do 0->1 , no benefit in disconnection
                }
            }
        }
    }
};

```

```

        if(!canreach(temp)){
            flag=true;
            return true; //disconnect ho gaya
        }
    }
}

if(!flag) return false;
return true;
};

```

Optimal Approach

We don't need to try every cell.

(S1) Do BFS/DFS once, travel along a path and make every travelled path as 0 (except first and last).

Original Path

1	1	1
0	1	1
0	0	1



After Travelling (Turned to 0)

1	0	0
0	1	0
0	0	1

Now if another path still remains, we were not able to disconnect it.

To see if a path still remains we need to do BFS/DFS again.

Reasoning: We are turning walked upon path to 0, because these are the critical cells, only turning them (1 → 0) can lead to disconnection.

```

class Solution {
public:

    vector<pair<int,int>> dir={{0,1},{1,0}};
    int n;
    int m;

    bool dfs(vector<vector<int>>& grid,int r,int c){

        if(r==n-1 && c==m-1) return true;
        grid[r][c]=0;
        for(auto &it:dir){
            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nc>=0 && nr<n && nc<m){
                if(grid[nr][nc]==1){
                    if(dfs(grid,nr,nc)){
                        return true;
                    }
                }
            }
        }

        return false;
    }

    bool isPossibleToCutPath(vector<vector<int>>& grid) {

        n=grid.size();
        m=grid[0].size();

        if(!dfs(grid,0,0)) return true; //already disconnect

        grid[0][0]=1;
        grid[n-1][m-1]=1; //restore src and dst

        if(!dfs(grid,0,0)) return true; //after eliminating crucial cell of one pa
        //crucial cell exist only on path itself

        return false;
    }
};

```